Difusão viscosa com OpenMP: schedule e collapse

Matheus Marinho 18 de outubro de 2026

O que foi feito

Simulei o movimento de um fluido com a equação de Navier-Stokes considerando **só a viscosidade**, sem pressão e sem forças externas. Validei a simulação com três testes e depois paralelizei com OpenMP, comparando `schedule(static)`, `schedule(dynamic)` e `schedule(guided)`, cada um com e sem `collapse(2)`.

Resultado: `static` sem `collapse` foi o melhor em todos os casos (até 42×). `dynamic` com `collapse(2)` foi **9× mais lento que o código sequencial**.

O modelo

Sem pressão e sem forças externas, sobra só a viscosidade. Cada componente da velocidade (u na horizontal, v na vertical) vira uma equação de **difusão**:

$$\partial u/\partial t = \nu \nabla^2 u \qquad \partial v/\partial t = \nu \nabla^2 v$$

- **Espaço:** malha n × n, com o laplaciano por **diferenças finitas**, usando os 4 vizinhos: `(cima + baixo + esquerda + direita − 4·centro) / h²`.
- **Tempo:** Euler explícito: `novo = atual + ν·Δt·∇²(atual)`.
- **Estabilidade:** esse método só é estável se `r = ν·Δt/h² ≤ 1/4`. Usei `r = 0,20`.
- **Bordas periódicas:** a malha "dá a volta" (o `%` no código).

> ***Como eu explico:*** *o laplaciano compara a célula com os vizinhos. Se ela está maior que eles, diminui; se está menor, aumenta. Por isso uma perturbação vai se espalhando e o campo fica uniforme, que é o efeito da viscosidade.*

Como funciona o código

parte	o que faz
`u`, `v` e `un`, `vn`	passo atual e próximo passo. O próximo é calculado lendo só o atual
`atualiza_celula(i, j)`	a conta de uma célula, com o `%` para dar a volta nas bordas
`passo(versao)`	um passo inteiro; as 7 versões têm o **mesmo laço**, só a diretiva muda
`simular`	repete os passos e troca os ponteiros (o próximo vira o atual), sem copiar dados
`medir`	menor tempo de 5 execuções, com `omp_get_wtime`
`memcmp`	confere se cada versão paralela deu **exatamente** o mesmo resultado do sequencial

> ***Como eu explico:*** *uso duas cópias porque cada célula precisa dos vizinhos antigos. Se eu escrevesse por cima, uma célula já atualizada entraria na conta da vizinha, e em paralelo isso seria condição de corrida. Com duas cópias, nenhuma thread escreve onde outra lê, então não preciso de `critical` nem `atomic`.*

Validação

Malha 1024 × 1024, 200 passos, sequencial:

teste	esperado	obtido
fluido parado (u = 0, v = 0)	continua parado	desvio = 0
velocidade constante (u = 2, v = −1)	não muda	desvio = 0
perturbação no centro	o pico diminui e se espalha	pico 2,000 → 1,002
soma de todas as células	se conserva (nada entra ou sai)	1048577 → 1048577

As 6 versões paralelas deram resultado **idêntico byte a byte** ao sequencial, em todas as quantidades de threads.

Conceitos (slide 37)

cláusula	como divide as iterações
`schedule(static)`	divide em blocos iguais **antes** de começar. Sem custo durante a execução
`schedule(dynamic)`	cada thread **pede** o próximo bloco quando termina o seu. Padrão: bloco de **1** iteração
`schedule(guided)`	como o dynamic, mas os blocos começam grandes e vão **diminuindo**
chunksize	tamanho do bloco: `schedule(dynamic, 64)`, por exemplo. Aqui usei o padrão
`collapse(2)`	junta os dois laços (i e j) em um só: 1024 × 1024 = 1.048.576 iterações para dividir

Resultados

NPAD, nó `r2n20` exclusivo, GCC 8.5.0, `-O2`, threads fixas em núcleos (`OMP_PLACES=cores`, `OMP_PROC_BIND=close`). Malha 1024 × 1024, 200 passos, menor tempo de 5 execuções. Sequencial: **0,83 s**. Tempo em segundos e speedup:

versão	16 threads	32 threads	64 threads	128 threads
`static`	**0,0545** (15,2×)	**0,0290** (28,6×)	**0,0196** (42,3×)	**0,0281** (29,6×)
`static` + `collapse(2)`	0,1062 (7,8×)	0,0543 (15,3×)	0,0310 (26,8×)	0,0295 (28,2×)
`dynamic`	0,2343 (3,5×)	0,3051 (2,7×)	0,3687 (2,3×)	0,8141 (1,0×)
`dynamic` + `collapse(2)`	7,7074 (0,11×)	7,3814 (0,11×)	7,4351 (0,11×)	8,9711 (0,09×)
`guided`	0,1387 (6,0×)	0,2091 (4,0×)	0,2662 (3,1×)	0,5473 (1,5×)
`guided` + `collapse(2)`	0,1536 (5,4×)	0,2161 (3,9×)	0,2755 (3,0×)	0,4919 (1,7×)

Análise

1. `static` é o melhor porque todas as células custam igual. Toda célula faz a mesma conta. Não há desbalanceamento para corrigir, então dividir em blocos iguais antes de começar é o ideal, e as threads não precisam combinar nada durante a execução. Com 16 threads a eficiência foi de 0,95.

2. `dynamic` piora com mais threads. Com bloco 1, cada linha é um pedido: 1024 pedidos por passo, todas as threads disputando o mesmo contador. Quanto mais threads, mais disputa: foi de 3,5× (16 threads) para 1,0× (128). E cada linha pega é de um lugar diferente da matriz, o que atrapalha o cache.

3. `guided` fica no meio. Começa com blocos grandes e faz menos pedidos que o `dynamic`, mas no fim os blocos ficam pequenos e a disputa volta. Melhor que `dynamic`, pior que `static`.

4. `dynamic` + `collapse(2)` é o pior caso. Com o `collapse`, são 1.048.576 iterações por passo, e com bloco 1 cada **célula** vira um pedido: mais de 200 milhões de pedidos na simulação. Pedir a célula custa mais do que calculá-la: 9× mais lento que o sequencial.

5. `collapse(2)` não ajudou. Ele serve quando o laço de fora tem poucas iterações para as threads. Aqui são 1024 linhas para no máximo 128 threads, então já sobra trabalho. O que ele traz é custo: o programa precisa recalcular `i` e `j` a partir de um índice só, a cada iteração. Com `static` e 16 threads, ficou 2× mais lento.

6. Com 128 threads o `static` piorou. São só 8 linhas por thread, e a cada um dos 200 passos as 128 threads precisam ser sincronizadas. O trabalho de cada uma ficou pequeno perto desse custo fixo, que é a sobrecarga de paralelização.

> ***Como eu explico:*** *o `static` ganhou porque toda célula custa a mesma coisa, então dividir igual antes de começar é o melhor. O `dynamic` só vale a pena quando as iterações têm custos diferentes. Aqui ele só adicionou o custo de pedir trabalho a cada linha. E com `collapse`, esse pedido passa a ser a cada célula, por isso ficou mais lento até que o sequencial.*

Conclusão

- **Use `static`** quando as iterações custam igual. É o caso deste problema.
- **`dynamic` e `guided`** servem para iterações com custos diferentes. Aqui só adicionaram custo. Com bloco 1, o custo de pedir trabalho supera o de calcular.
- **`collapse`** serve quando o laço de fora tem iterações de menos para as threads. Não é o caso: 1024 linhas já bastam.

Código

difusao_viscosa.c · 196 linhas

```c
#include <math.h>
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static int n;                    /* a malha tem n x n celulas */
static double fator;             /* viscosidade * dt / h^2 */
static double *u, *v, *un, *vn;  /* passo atual (u, v) e proximo (un, vn) */

/* Calcula a celula (i, j) do proximo passo. Bordas periodicas: o % faz a
   malha dar a volta. */
static inline void atualiza_celula(int i, int j)
{
    int cima = (i + n - 1) % n, baixo = (i + 1) % n;
    int esq = (j + n - 1) % n, dir = (j + 1) % n;
    int c = i * n + j;

    double lap_u = u[cima * n + j] + u[baixo * n + j] + u[i * n + esq] + u[i * n + dir]
                 - 4.0 * u[c];
    double lap_v = v[cima * n + j] + v[baixo * n + j] + v[i * n + esq] + v[i * n + dir]
                 - 4.0 * v[c];

    un[c] = u[c] + fator * lap_u;
    vn[c] = v[c] + fator * lap_v;
}

/* Um passo da simulacao. As sete versoes tem o mesmo laco; so a diretiva muda. */
static void passo(int versao)
{
    switch (versao) {
    case 0:
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                atualiza_celula(i, j);
        break;
    case 1:
#pragma omp parallel for schedule(static)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                atualiza_celula(i, j);
        break;
    case 2:
#pragma omp parallel for schedule(static) collapse(2)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                atualiza_celula(i, j);
        break;
    case 3:
#pragma omp parallel for schedule(dynamic)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                atualiza_celula(i, j);
        break;
    case 4:
#pragma omp parallel for schedule(dynamic) collapse(2)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                atualiza_celula(i, j);
        break;
    case 5:
#pragma omp parallel for schedule(guided)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                atualiza_celula(i, j);
        break;
    case 6:
#pragma omp parallel for schedule(guided) collapse(2)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                atualiza_celula(i, j);
        break;
    }
}

/* Avanca a simulacao. Depois de cada passo, o proximo vira o atual: so os
   ponteiros sao trocados, sem copiar dados. */
static void simular(int versao, int passos)
{
    for (int t = 0; t < passos; t++) {
        passo(versao);
        double *tmp = u; u = un; un = tmp;
        tmp = v; v = vn; vn = tmp;
    }
```

```c
}

static void inicia(double u0, double v0)
{
    for (int k = 0; k < n * n; k++) {
        u[k] = u0;
        v[k] = v0;
    }
}

static double maior_desvio(const double *campo, double referencia)
{
    double maior = 0.0;
    for (int k = 0; k < n * n; k++)
        if (fabs(campo[k] - referencia) > maior)
            maior = fabs(campo[k] - referencia);
    return maior;
}

static double soma(const double *campo)
{
    double s = 0.0;
    for (int k = 0; k < n * n; k++)
        s += campo[k];
    return s;
}

static double maximo(const double *campo)
{
    double maior = campo[0];
    for (int k = 1; k < n * n; k++)
        if (campo[k] > maior)
            maior = campo[k];
    return maior;
}

/* Menor tempo entre algumas execucoes da simulacao com a perturbacao central. */
static double medir(int versao, int passos, int execucoes)
{
    double melhor = 1e30;
    for (int e = 0; e < execucoes; e++) {
        inicia(1.0, 0.0);
        u[(n / 2) * n + n / 2] += 1.0;

        double inicio = omp_get_wtime();
        simular(versao, passos);
        double tempo = omp_get_wtime() - inicio;

        if (tempo < melhor)
            melhor = tempo;
    }
    return melhor;
}

int main(int argc, char *argv[])
{
    n = (argc > 1) ? atoi(argv[1]) : 1024;
    int passos = (argc > 2) ? atoi(argv[2]) : 200;
    int threads = (argc > 3) ? atoi(argv[3]) : omp_get_max_threads();
    int execucoes = 5;

    double h = 1.0 / n, viscosidade = 0.01, r = 0.20;
    double dt = r * h * h / viscosidade;   /* Euler explicito: estavel se r <= 1/4 */
    fator = viscosidade * dt / (h * h);

    omp_set_num_threads(threads);
    u = malloc(n * n * sizeof(double));
    v = malloc(n * n * sizeof(double));
    un = malloc(n * n * sizeof(double));
    vn = malloc(n * n * sizeof(double));
    double *referencia = malloc(n * n * sizeof(double));

    printf("Malha %d x %d, %d passos, dt = %.3e, r = %.2f, %d threads\n\n",
           n, n, passos, dt, r, threads);

    printf("Validacao (sequencial)\n");
    inicia(0.0, 0.0);
    simular(0, passos);
    printf("1) fluido parado (u=0, v=0): maior |u| = %.1e, maior |v| = %.1e\n",
           maior_desvio(u, 0.0), maior_desvio(v, 0.0));

    inicia(2.0, -1.0);
    simular(0, passos);
    printf("2) campo constante (u=2, v=-1): desvio u = %.1e, desvio v = %.1e\n",
           maior_desvio(u, 2.0), maior_desvio(v, -1.0));

    inicia(1.0, 0.0);
    u[(n / 2) * n + n / 2] += 1.0;
    double pico0 = maximo(u), soma0 = soma(u);
```

```c
    simular(0, passos);
    printf("3) perturbacao central: pico %.6f -> %.6f, soma %.4f -> %.4f\n\n",
           pico0, maximo(u), soma0, soma(u));

    const char *nomes[7] = { "sequencial", "static", "static + collapse(2)",
                             "dynamic", "dynamic + collapse(2)",
                             "guided", "guided + collapse(2)" };

    printf("Desempenho (menor tempo de %d execucoes)\n", execucoes);
    printf("versao,tempo_s,speedup,confere\n");
    double t_seq = medir(0, passos, execucoes);
    memcpy(referencia, u, n * n * sizeof(double));
    printf("%s,%.4f,1.00,-\n", nomes[0], t_seq);

    for (int versao = 1; versao <= 6; versao++) {
        double t = medir(versao, passos, execucoes);
        int confere = memcmp(u, referencia, n * n * sizeof(double)) == 0;
        printf("%s,%.4f,%.2f,%s\n", nomes[versao], t, t_seq / t, confere ? "sim" : "NAO");
    }

    free(u); free(v); free(un); free(vn); free(referencia);
    return 0;
}
```